SPRING BOOT - MAVEN

esprit
Se former autrement
HONORIS UNITED UNIVERSITIES



Plan du Cours

- Définition Spring Boot
- Avantages Spring Boot
- Définition Maven
- Etapes de construction de projet
- Arborescence du projet
- Gestion et portée des dépendances
- TP Spring Boot

SPRING BOOT

- **Spring Boot** est le projet principal de Spring : https://spring.io/projects
- Il simplifie le démarrage et le développement de nouvelles applications Spring.
- Il diminue énormément le temps de développement et augmente la productivité.
- Spring Boot permet l'utilisation de conteneurs embarqués (embedded containers) : web Server fourni par Spring Boot (comme Tomcat …).
- Avec Spring Boot, les configurations de Spring sont diminuées.

AVANTAGES SPRING BOOT

- Spring Boot offre trois avantages incontournables :

➢ La gestion des configurations
➢ Le serveur est embarqué
➢ La gestion des dépendances

AVANTAGES - Gestion des configurations

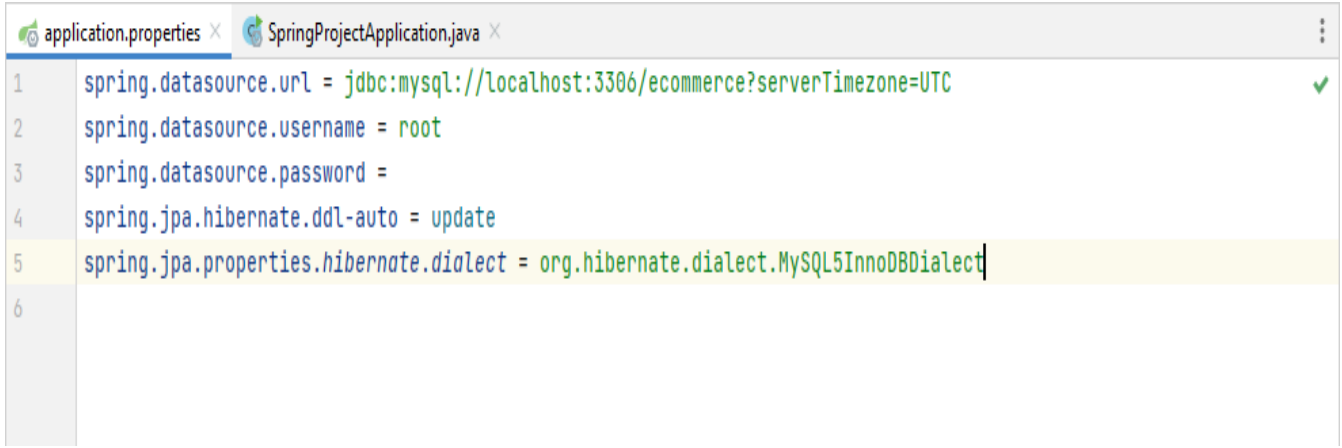
- Spring Boot facilite **la gestion des configurations** en centralisant les configurations dans un seul fichier. Ainsi en se focalisant sur le métier au lieu de la configuration, le développeur devient beaucoup plus productif.

Avant Spring Boot

web.xml
persistance.xml
dispatcher.xml
log4J.xml
……

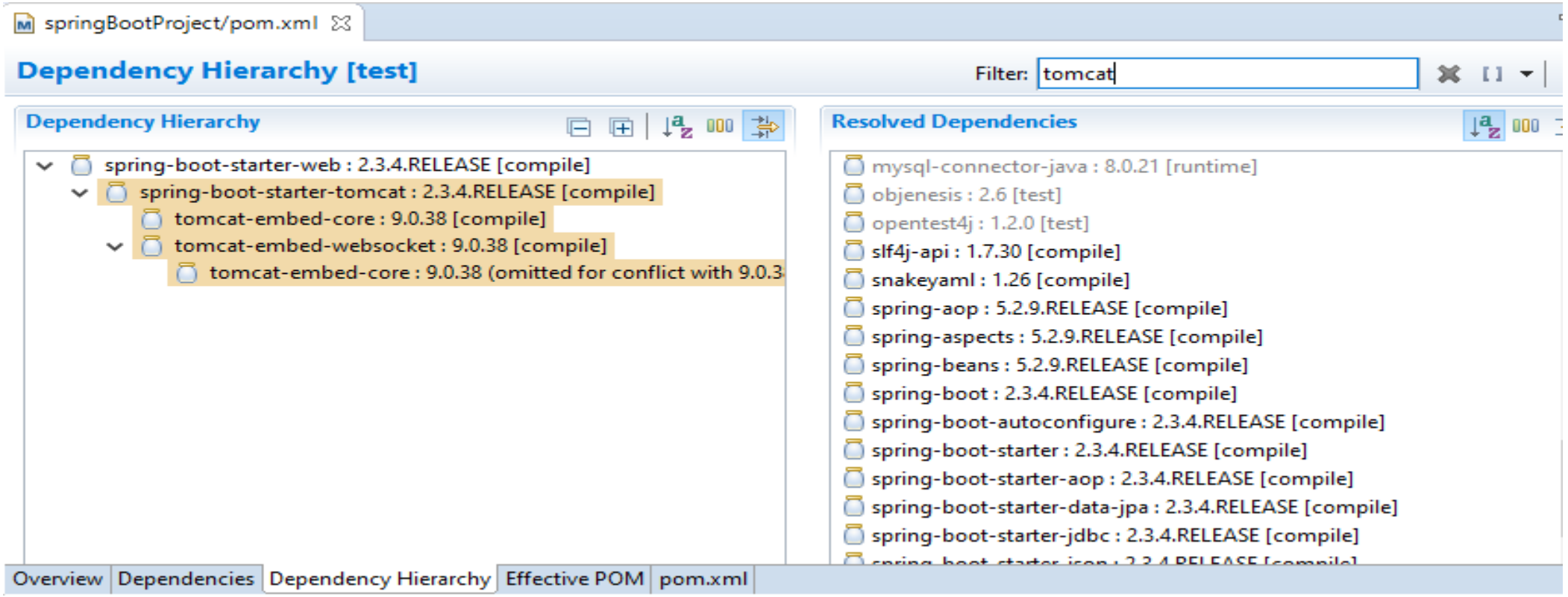
Avec Spring Boot

Un seul fichier application.properties



AVANTAGES - Serveur Embarquée

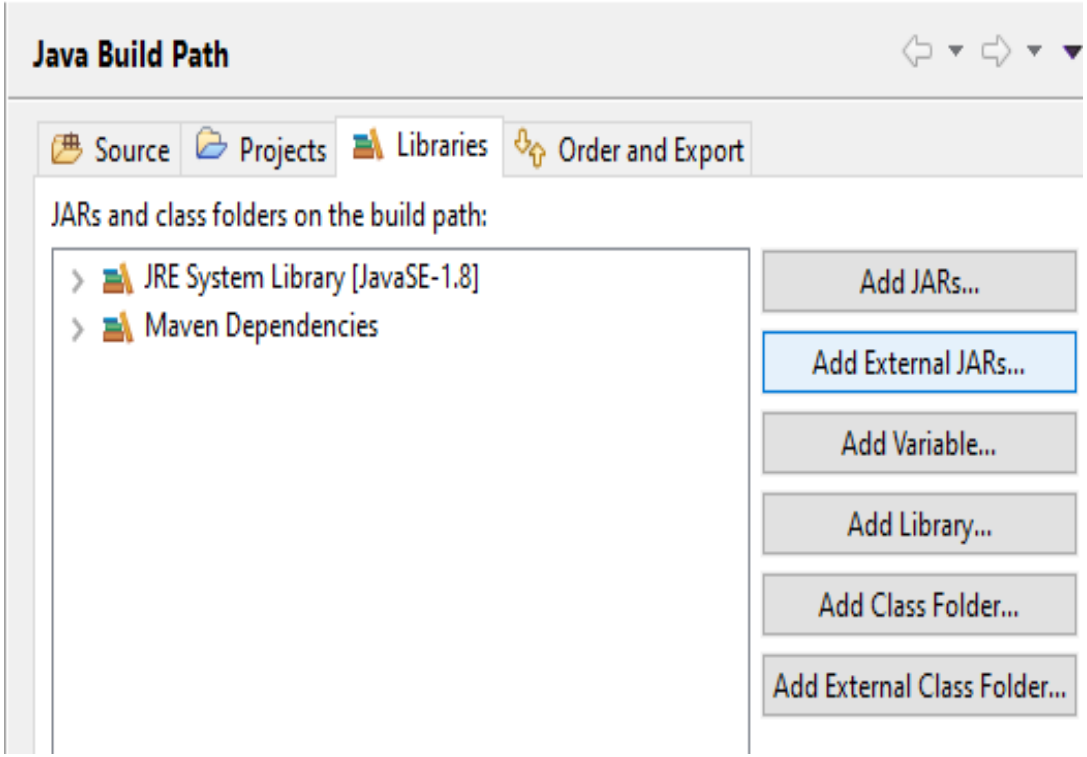
- **Spring Boot** fournit des serveurs intégrés (Embedded HTTP servers) comme Tomcat, Jetty .... afin de développer et de tester des applications web facilement.
- En lançant le projet, le jar du Tomcat dézippe et se lance.



AVANTAGES - Gestion des dépendances

- Spring Boot facilite **la gestion des dépendances** pour commencer un projet Spring.

Avant Spring et Spring Boot



Avec Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

AVANTAGES - Gestion des dépendances

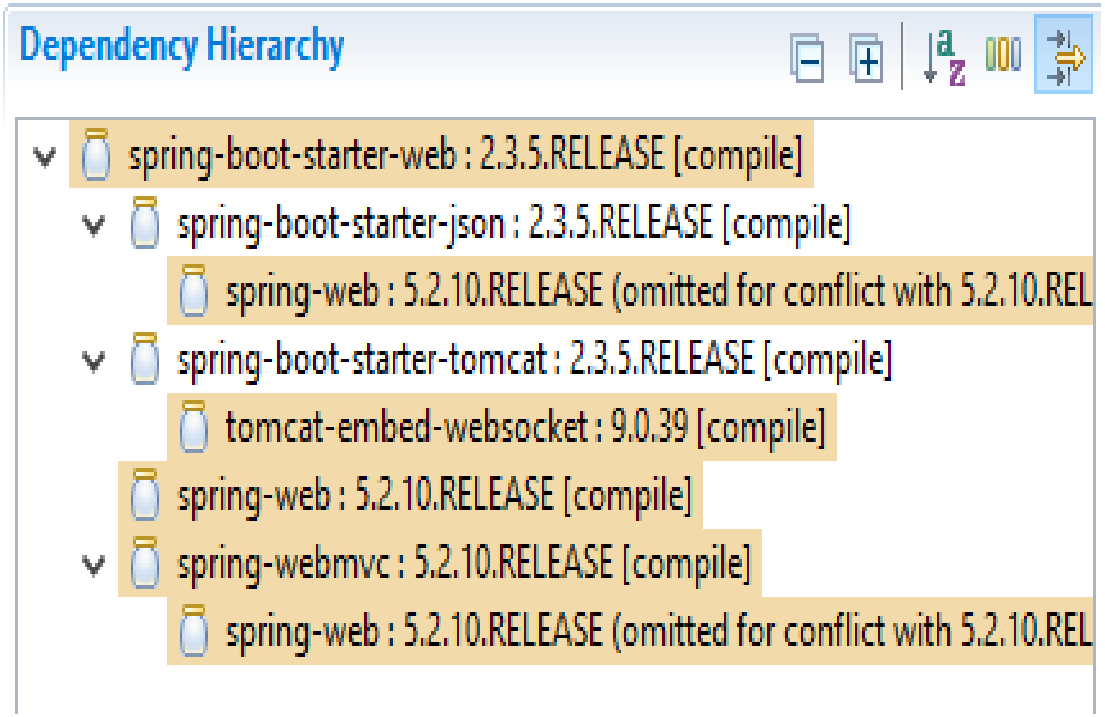
- Spring Boot facilite **la gestion des dépendances** grâce notamment à l'utilisation des **starters** (une seule dépendance qui ramène toutes les dépendances nécessaires).

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

AVANTAGES - Gestion des dépendances

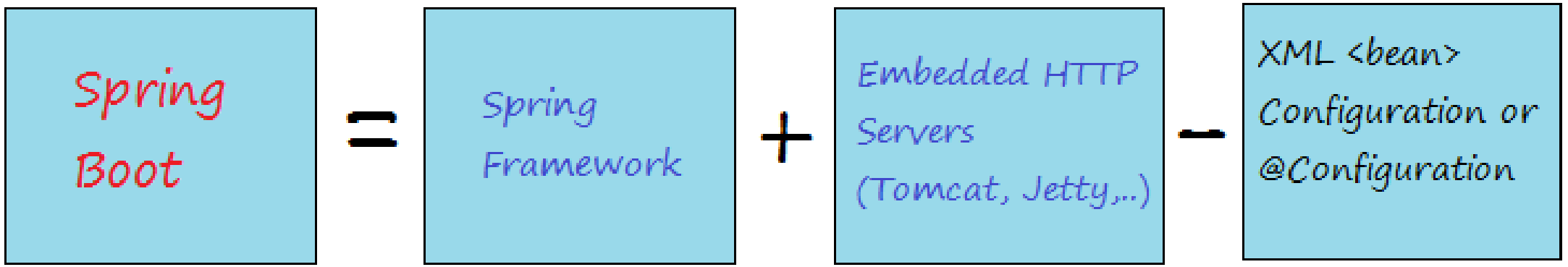
- Un **starter** va apporter à votre projet un ensemble de dépendances, communément utilisées pour un type de projet donné.
- Les starters facilitent la **gestion des versions**. Plus besoin de chercher quelles versions sont compatibles afin de les ajouter une à une dans le pom.xml.




```
<parent>
<groupId>
org.springframework.boot
</groupId>
<artifactId>
spring-boot-starter-parent</artifactId>
<version>2.7.3.RELEASE</version>
<relativePath/>
</parent>
```

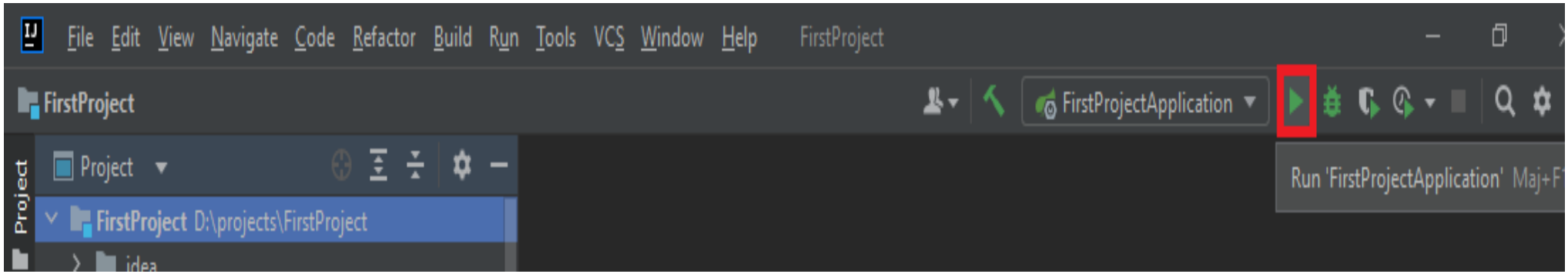
AVANTAGES SPRING BOOT

Spring Boot peut s'expliquer simplement par l'illustration ci-dessous:



SPRING BOOT

- ***Comment démarrer un projet Spring Boot :***



- Spring Boot fournit beaucoup de plugins afin de développer et tester des applications Spring Boot rapidement en utilisant les outils de Build comme Maven et Gradle.

DEFINITION MAVEN

Maven est un outil pour le fciltiter la **construction** de projets (Build, Dépendances, …) et **l'automatisation** des différentes phases projets (Tests, Packaging, …). Maven permet donc :

- Gestion des builds (compilation)
- La gestion des dépendances (*.jar), se basant sur des conventions (POM)
- Lancement des tests (Unitaire, de Qualité de Code, …)
- Génération des livrables

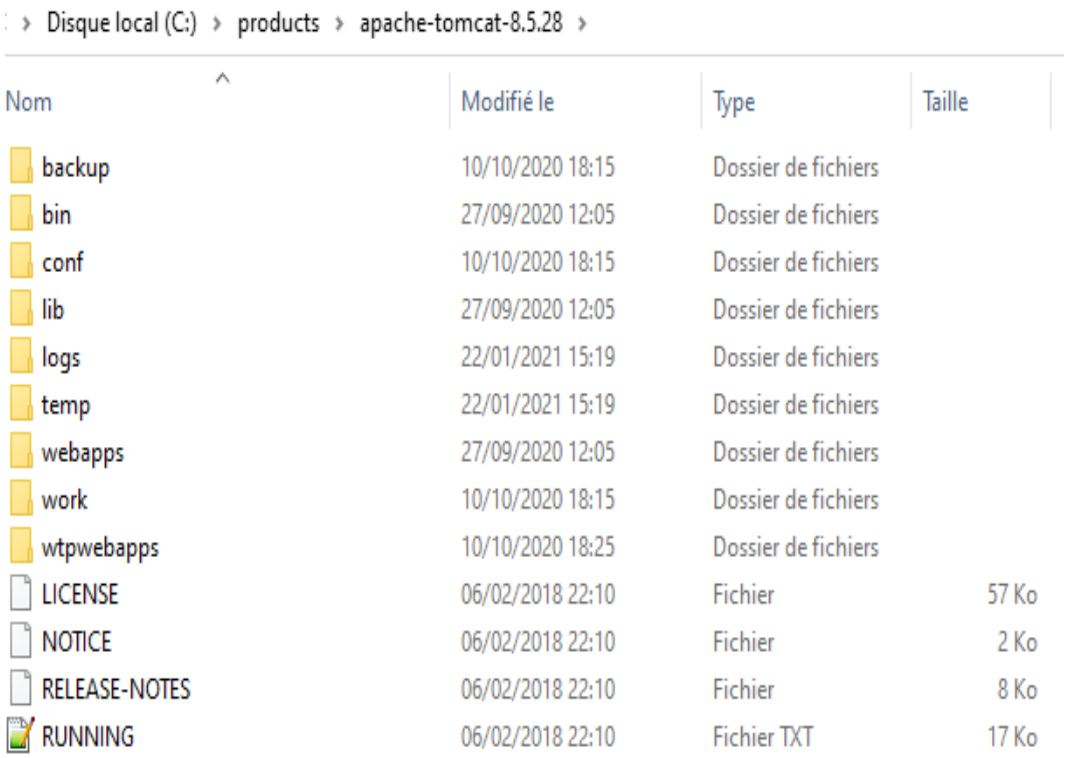
=> Automatisation de tâches récurrentes => Intégration Continue (DevOps)



INSTALLATION DE MAVEN

- Maven peut être installé :

➢ **1-** En mode **Standalone**
Exemple: Utiliser Maven dans l'intégration continue (domaine DevOps)

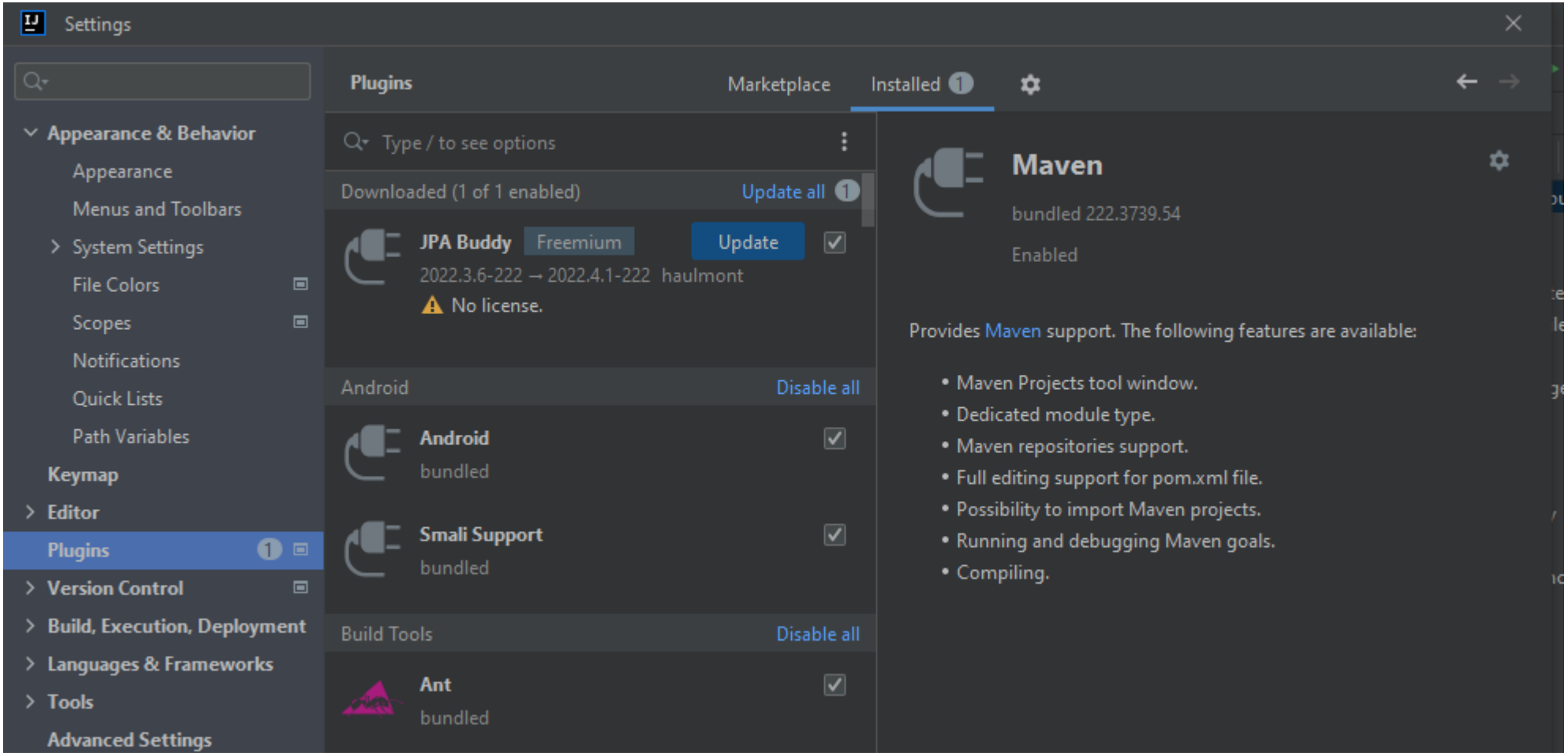


➢ **2-** En tant que **Plugin** : Par défaut, Intellij intègre un plugin Maven

➢ Nous allons utiliser Maven en tant que Plugin.

INSTALLATION DE MAVEN

- En mode plugin :
- Cliquer sur File => Settings => Plugins



PREMIER PROJET MAVEN

Un projet est caractérisé par :

- **project** : Balise racine de tous les fichiers pom.xml.
- **modelVersion** : Version de POM utilisée.
- **groupId** : Identifier un groupe qui a créé le projet. Ex: org.apache.
- **artifactId** : Nom unique utilisé pour nommer l'artifact à construire.
- **packaging** : Type de packaging du projet (ex. : JAR, WAR, EAR...).
- **archetype** : Template de Projet.
- **name** : Nom du projet.
- **description** : Description du projet.

GESTION DES VERSIONS DU PROJET

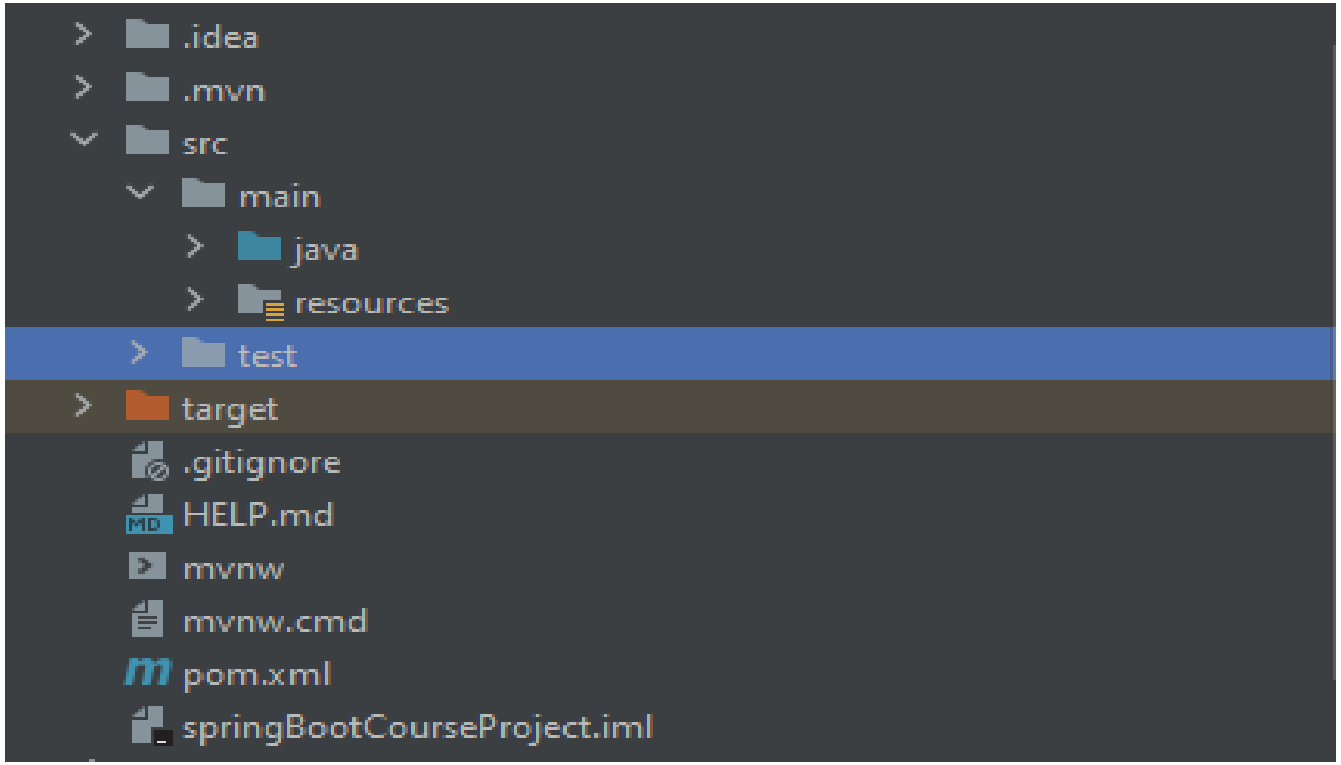
Exercice

Soit les versions du projet suivantes :
1.0.0
1.1.0
2.0.0
2.0.1
3.0.0

A quoi correspond chaque numérotation ?
Release majeure
Release mineure
Patch (Correctif)

ARBORESCENCE DU PROJET

- Maven est basé sur la notion de **convention over configuration (arborescence prédéfinie)** ce qui le différencie par rapport à ses concurrents (Gradle, Ant) où une configuration supplémentaire est requise.



ARBORESCENCE DU PROJET

- **pom.xml** : le fichier de configuration du projet
- **/src** : code source et fichiers source principaux
- **/src/main/java** : code source java
- **/src/main/resources** : fichiers de ressources (images, fichiers config...)
- **/src/main/webapp** : webapp du projet
- **/src/test** : fichiers de test
- **/src/test/java** : code source Java de test
- **/src/test/resources** : fichiers de ressources de test
- **/target** : fichiers résultat, les binaires (du code et des tests), les packages générés et les résultats des tests

Fichier POM.XML

- Permet de spécifier les dépendances dont le projet a besoin.
- Spécifie l'emplacement de l'artefact du projet (groupId, ArtifactId, version)

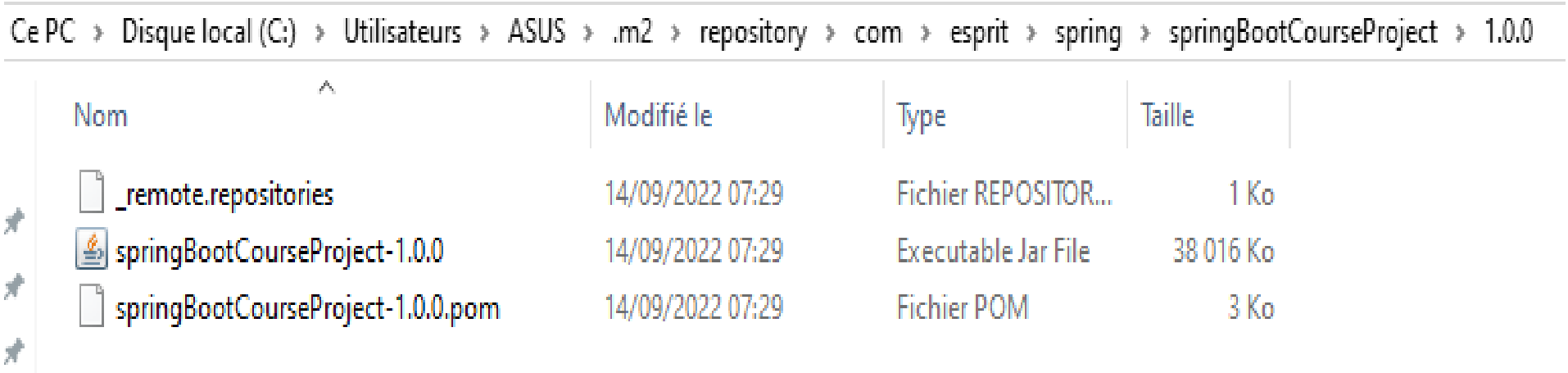
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.esprit.spring</groupId>
<artifactId>springBootCourseProject</artifactId>
<version>1.0.0</version>
<name>springBootCourseProject</name>
<description>springBootCourseProject</description>
<dependencies><dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-core</artifactId>
<version>2.14.0</version>
</dependency></dependencies>
</project>
```

Etapes de construction du projet

- **1 mvn compile** : Créer les .class
- **2 mvn test** : Jouer les tests unitaires
- **3 mvn package** : Créer le livrable dans target.
- **4 mvn install** : Copier le livrable dans le Repository local : ~\.m2\repository\...
- **5 mvn deploy** : Copier le livrable sur le repository distant (Exemple : Nexus)
- **mvn clean** : Supprimer le contenu du dossier target.

Etapes de construction du projet

- Emplacement du livrable :
  {emplacement Repository}/groupId/artifactId/version
- Nom du package (jar en général) : {artifactId}-{version}.{package}

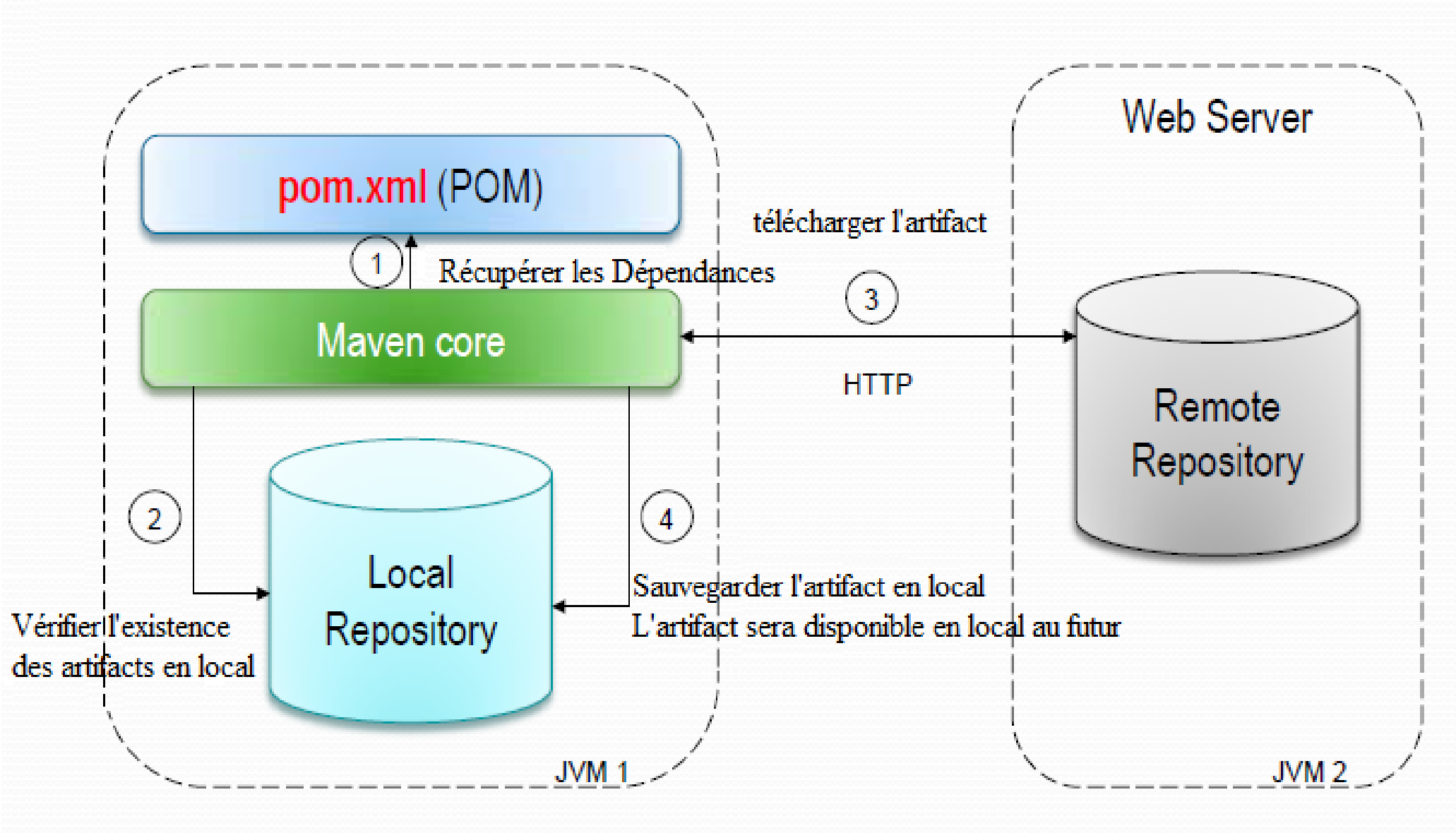


Gestion des dépendances

Pour ajouter une dépendance, il suffit de chercher la dépendance en question dans le **mvnRepository** (https://mvnrepository.com/) et l'inclure dans le pom.xml sous la balise **<dependencies>** comme suit (Spring Boot fera cela à notre place):

```
<dependencies>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.23</version>
</dependency>
</dependencies>
```

Gestion des dépendances



Dépôts Maven

- Maven s'appuie sur les dépôts (repositories) pour stocker les jars de dépendances et des livrables.

- Il y a deux types de repositories :

➢ **Local** : Le dépôt local se trouve par défaut sous l'arborescence **%path_dossier_M2%\repository**

➢ **Remote** (Distant) :
  ➢ Central : dépôt public Maven accessible via https://mvnrepository.com/
  ou
  ➢ Internal (Private) : dans les serveurs dédiés à l'entreprise (pour des raisons de sécurité)

- Les dépôts sont organisés en groupes, artefacts et versions

TP - Spring Boot- Maven

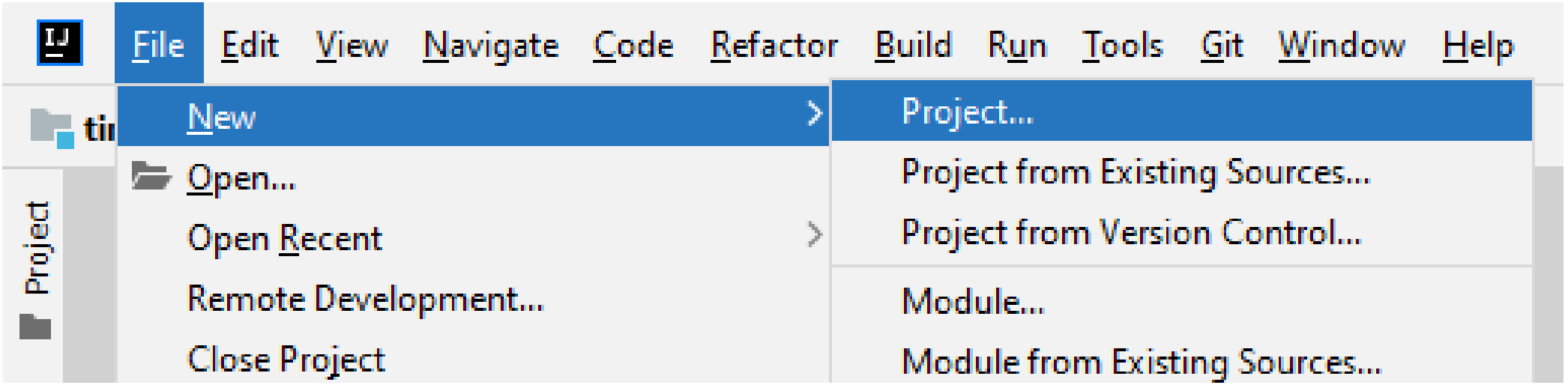
- Nous allons dans ce TP créer notre premier projet **Spring Boot**.
- Nous allons découvrir les différentes commandes liées au cycle de vie Maven et les appliquer sur le projet spring boot déjà créé.

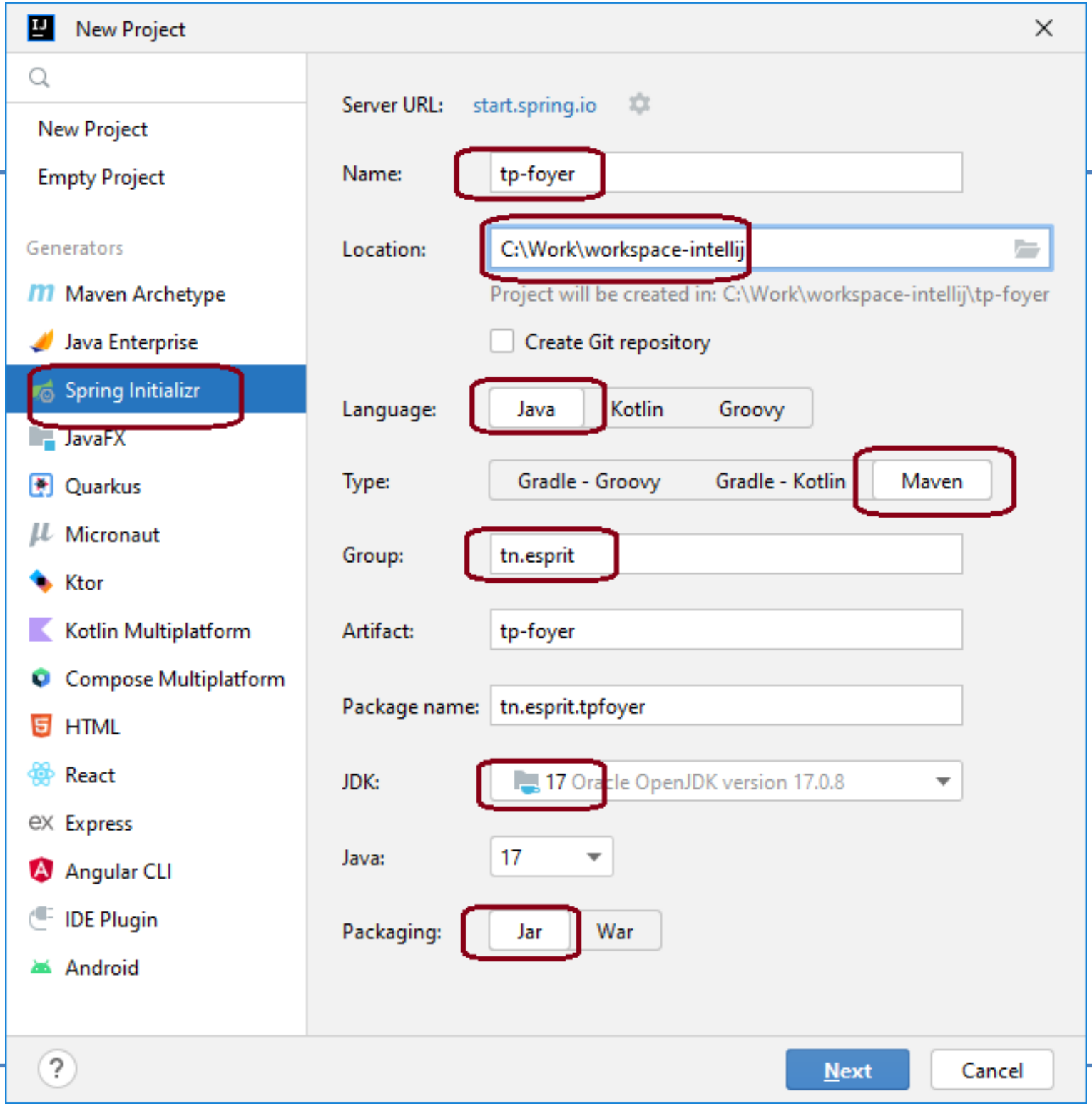
- Ces commandes Maven seront appelés et utilisés implicitement par Spring Boot, par la suite.

- Les étapes seront décrites dans les slides suivants :

TP - Spring Boot- Maven

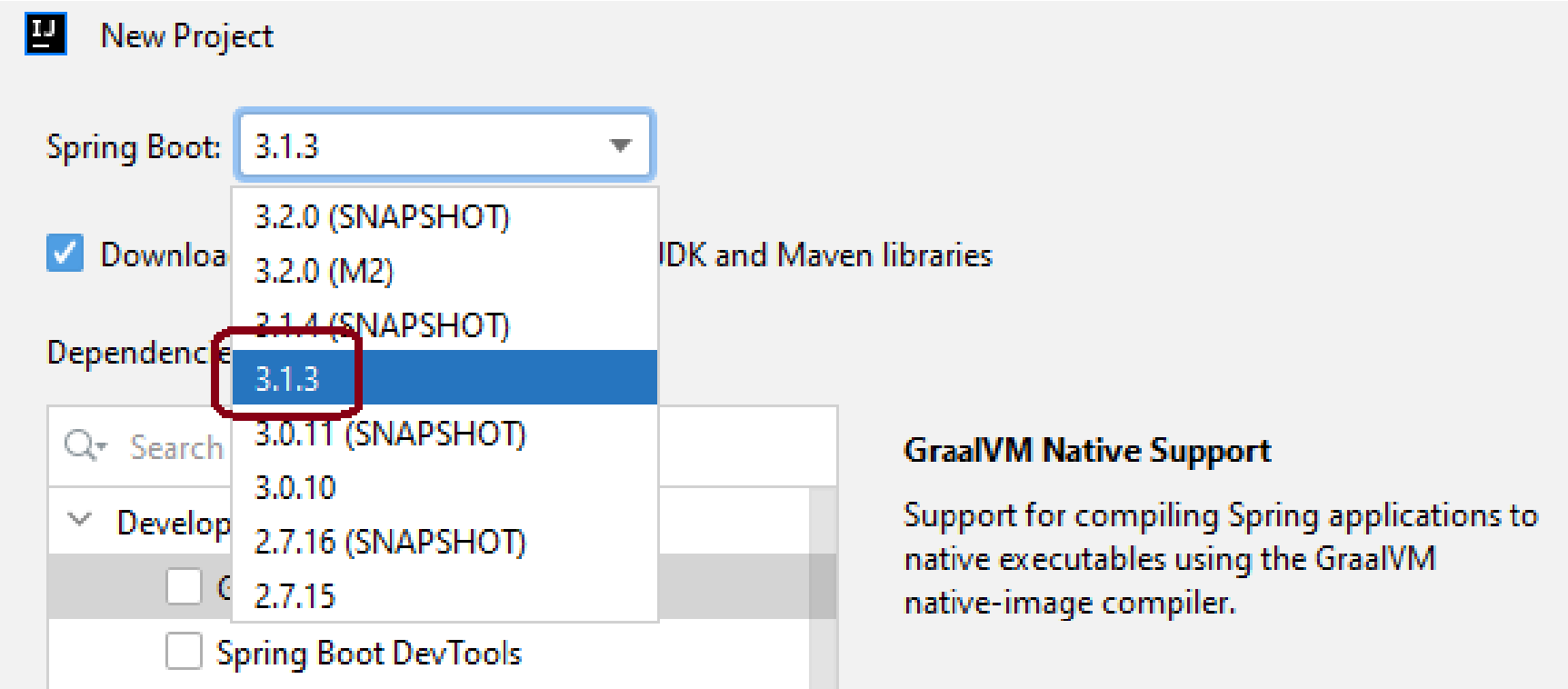
- Création d'un projet Spring Boot avec IntelliJ :




New Project
New Project
Empty Project
Generators
Maven Archetype
Java Enterprise
Spring Initializr
JavaFX
Quarkus
Micronaut
Ktor
Kotlin Multiplatform
Compose Multiplatform
HTML
React
Express
Angular CLI
IDE Plugin
Android
Server URL: start.spring.io
Name: tp-foyer
Location: C:\Work\workspace-intellij
Project will be created in: C:\Work\workspace-intellij\tp-foyer
Create Git repository
Language: Java Kotlin Groovy
Type: Gradle - Groovy Gradle - Kotlin Maven
Group: tn.esprit
Artifact: tp-foyer
Package name: tn.esprit.tpfoyer
JDK: 17 Oracle OpenJDK version 17.0.8
Java: 17
Packaging: Jar War
Next
Cancel

TP - Spring Boot- Maven

- Choix de la version 3 stable de Spring Boot :

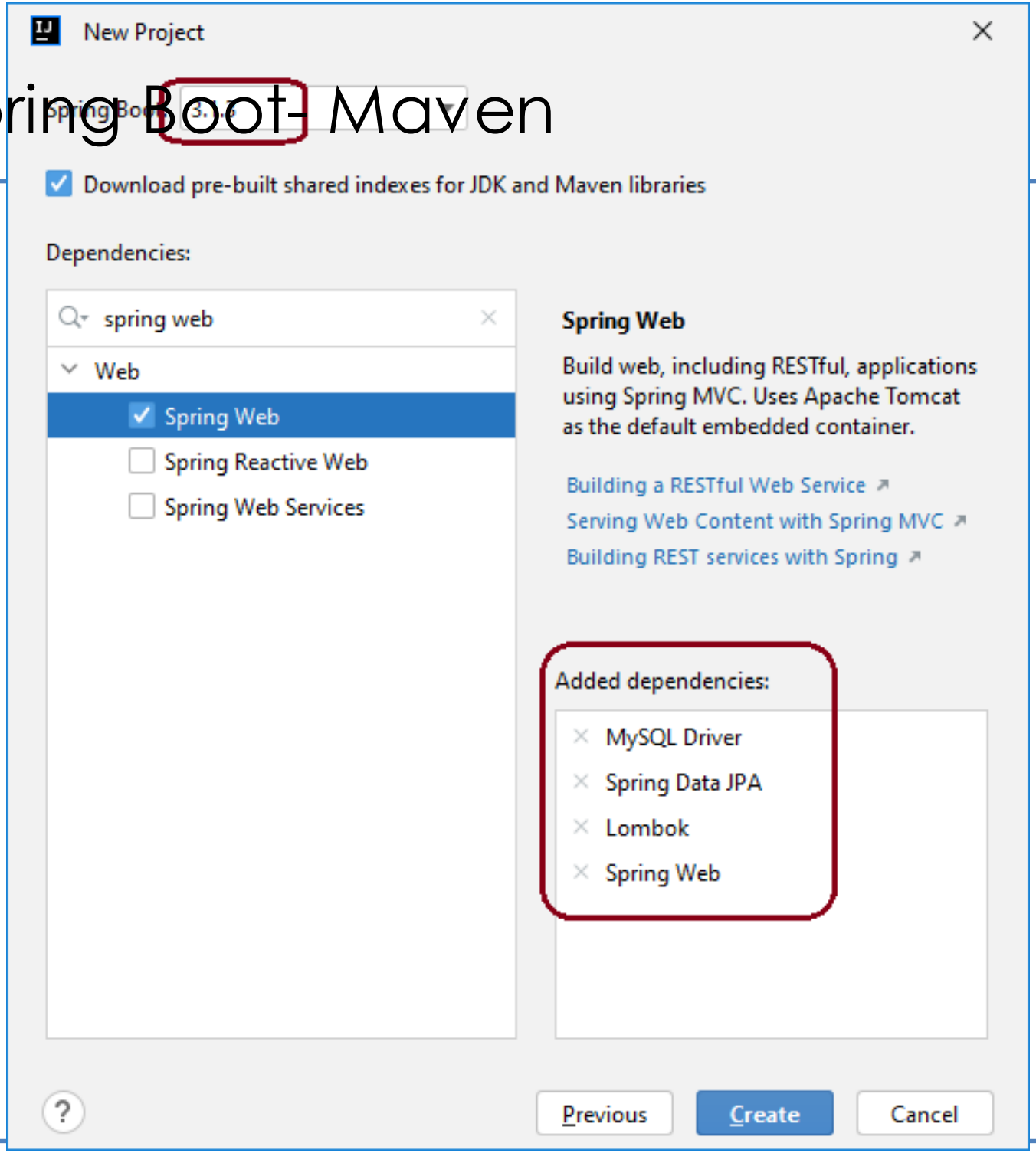


TP - Spring Boot- Maven

Choix des dépendances nécessaires pour notre projet Spring Boot **tp-foyer** :

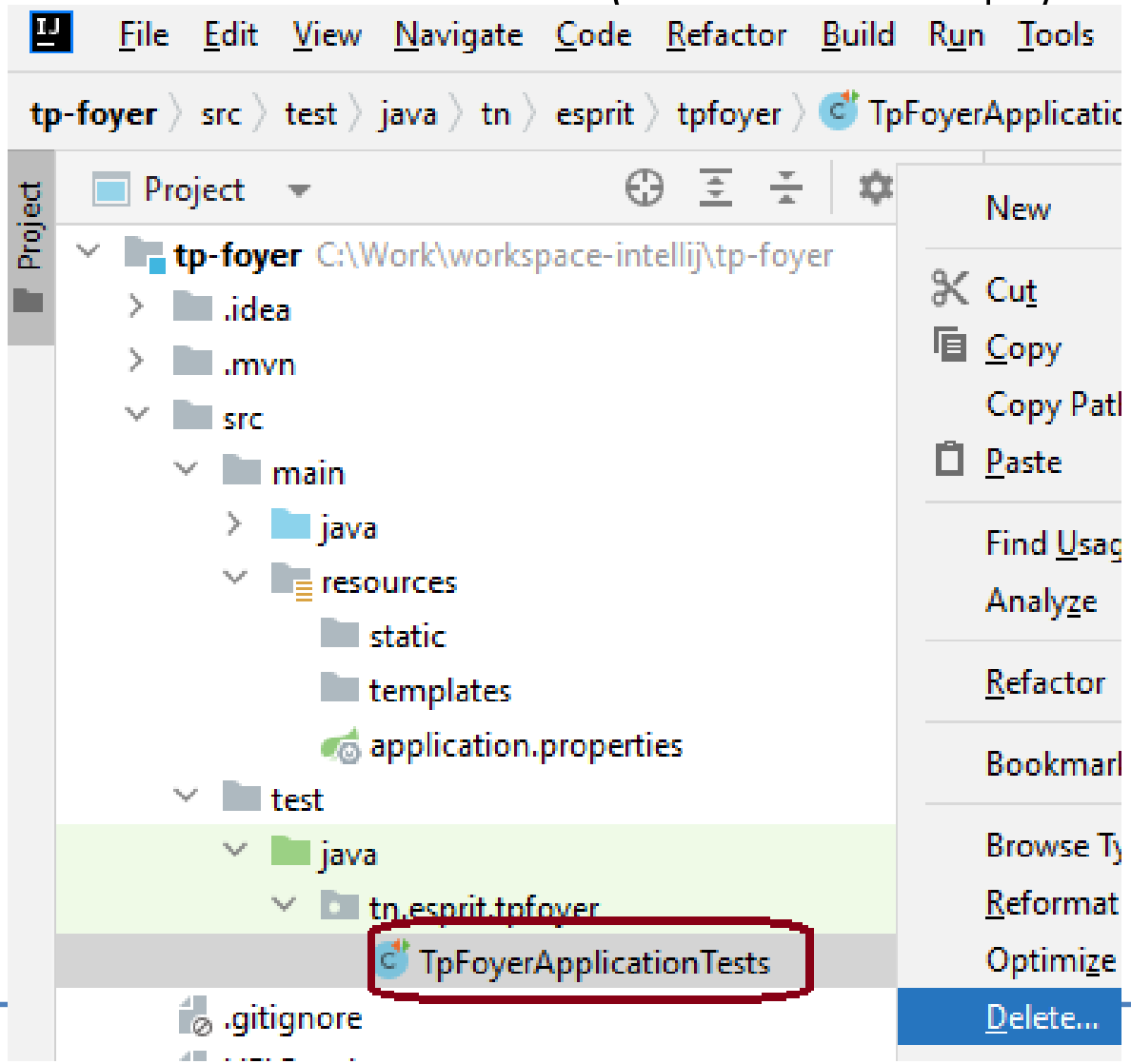
- **MySQL Driver**
- **Spring Data JPA**
- **Lombok**
- **Spring Web**

Cliquer ensuite sur « Create »
Attendez le téléchargement de toutes les dépendances par Maven
(Une bonne connexion Internet est nécessaire).



TP - Spring Boot- Maven

- Supprimer **la classe de test** pour éviter les erreurs lors de l'appel des commandes Maven (car « Maven install » par exemple essaiera de lancer les tests unitaires). Les tests unitaires seront vus en 5ème années (Module DeVOps) :



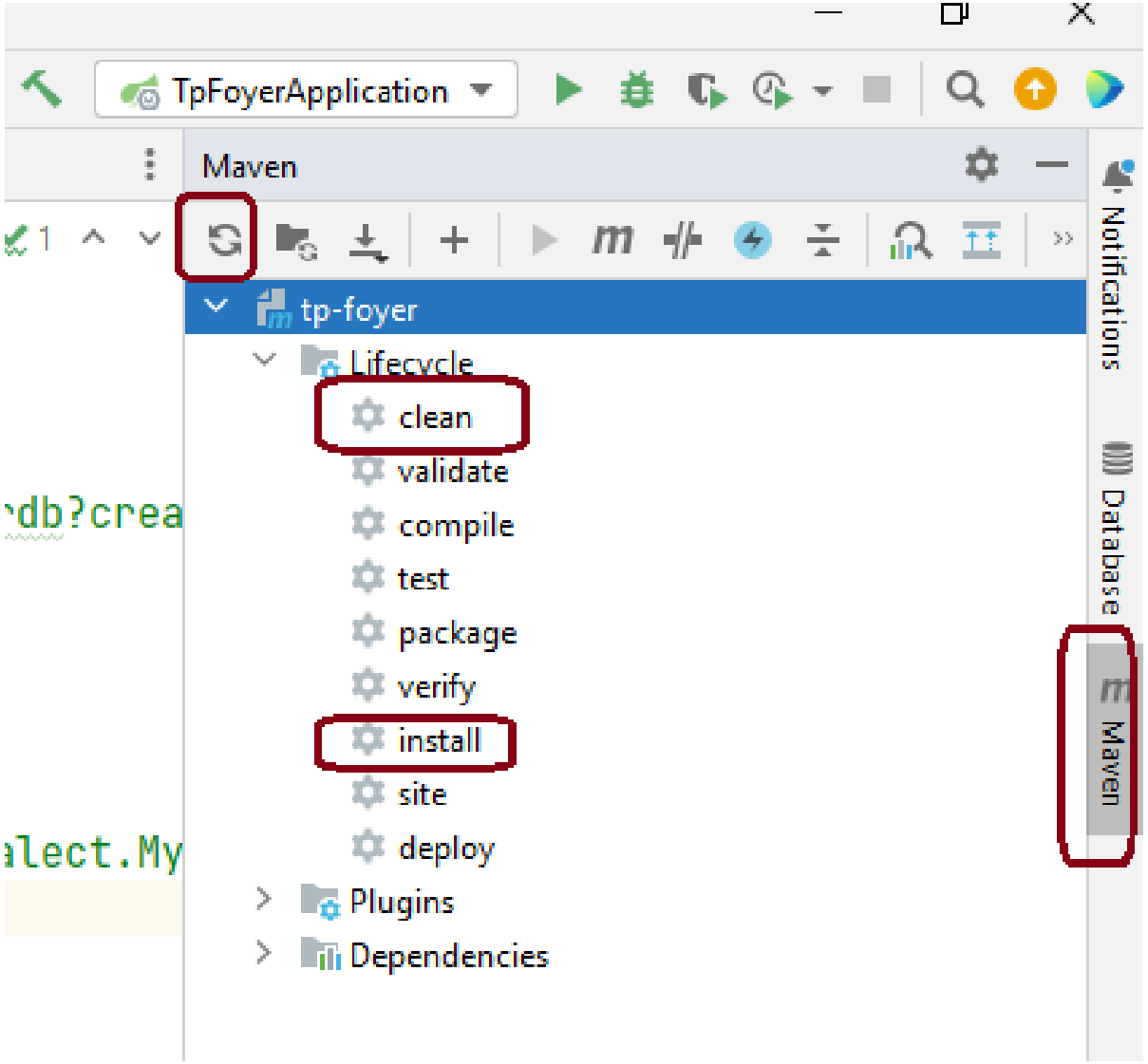
TP - Spring Boot- Maven

- Ajouter les properties suivantes pour éviter les erreurs lors du lancement des commandes Maven (Comme il y a la dépendance Spring Data JPA, Spring Boot vérifiera s'il y a une base de données de configurée) dans **application.properties** :

```
spring.datasource.url=jdbc:mysql://localhost:3306/tpfoyerdb?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

TP - Spring Boot- Maven

Tester les différents commandes maven



Prochaine Séance

Prochaine Séance : **3- Spring Data JPA – Entités** :

Création de vos premières entités avec Spring